

UNIVERSIDAD PRIVADA DOMINGO SAVIO

Diseño Web II

ENTREGA 4A

Página de Perfil de Usuario

Fase 2 — Gestión de Usuarios y Materias

Proyecto: gestión-académica

Prerrequisito: Fase 1 completada (Entregas 1A – 3B)

Abril 2026

Objetivos de Aprendizaje

Al completar esta entrega, el estudiante será capaz de:

- Definir esquemas de validación con Zod para formularios complejos.
- Integrar React Hook Form con Zod usando el resolver de @hookform/resolvers.
- Construir un formulario de edición de perfil que carga datos existentes y los actualiza via API.
- Usar notificaciones toast (Sonner) para feedback al usuario.
- Crear un componente reusable StatsCard para los dashboards.

Prerrequisitos

- ✓ Fase 1 completa (Entregas 1A – 3B)
- ✓ API Route GET/PUT /api/auth/perfil creada en la Entrega 3B
- ✓ Dependencias zod, react-hook-form y @hookform/resolvers instaladas

Teoría: Zod + React Hook Form

En la Fase 1 construimos formularios de login y registro con useState directo. Eso funciona para formularios simples, pero a medida que los formularios crecen en complejidad (más campos, validaciones, estados), el código se vuelve difícil de mantener. Por eso introducimos dos librerías complementarias:

¿Qué es Zod?

Zod es una librería de validación de esquemas. Defines las reglas de validación en un solo lugar y Zod las aplica automáticamente. Además, Zod genera tipos TypeScript a partir del esquema, así no duplicas la definición de tipos.

Ejemplo: Zod básico

```
// Ejemplo básico de Zod
import { z } from "zod"

// Definir esquema
const esquemaUsuario = z.object({
  nombre: z.string().min(2, "El nombre debe tener al menos 2 caracteres"),
  email: z.string().email("Email no válido"),
  edad: z.number().min(18, "Debes ser mayor de edad"),
})
```

```
// Zod genera el tipo TypeScript automáticamente
type Usuario = z.infer<typeof esquemaUsuario>
// Resultado: { nombre: string; email: string; edad: number }

// Validar datos
const resultado = esquemaUsuario.safeParse({ nombre: "A", email: "mal", edad: 15 })
// resultado.success = false
// resultado.error contiene los mensajes de error de cada campo
```

¿Qué es React Hook Form?

React Hook Form (RHF) maneja el estado del formulario de forma eficiente. A diferencia de useState para cada campo, RHF usa refs internamente, lo que significa menos re-renders y mejor rendimiento.

Comparación de rendimiento

```
// Sin React Hook Form (muchos re-renders):
const [nombre, setNombre] = useState("") // re-render al escribir
const [email, setEmail] = useState("") // re-render al escribir
const [telefono, setTelefono] = useState("") // re-render al escribir
// Cada tecla presionada causa un re-render de TODO el componente

// Con React Hook Form (mínimos re-renders):
const { register, handleSubmit } = useForm()
// register conecta el input directamente con un ref
// Solo hace re-render cuando es necesario (submit, error)
```

¿Cómo se conectan?

El paquete @hookform/resolvers actúa como puente entre ambas librerías. Le dice a React Hook Form: "usa este esquema de Zod para validar los datos antes de hacer submit".

Flujo de validación

FLUJO DE VALIDACIÓN:

1. Usuario escribe en un campo
 - ↳ React Hook Form captura el valor (sin re-render)
2. Usuario hace clic en "Guardar"
 - ↳ RHF ejecuta handleSubmit()
 - ↳ El resolver de Zod valida TODOS los campos
3. Si hay errores:
 - ↳ Zod retorna mensajes de error por campo
 - ↳ RHF los muestra debajo de cada input (re-render mínimo)
4. Si todo es válido:
 - ↳ Se ejecuta la función onSubmit con los datos tipados

Definir el esquema de validación del perfil

Creemos un archivo dedicado para el esquema Zod del perfil. Esto permite reutilizar las reglas de validación tanto en el formulario como en la API Route.

Crea el archivo **lib/validations/perfil.ts**:

```
Archivo: lib/validations/perfil.ts
import { z } from "zod"

export const perfilSchema = z.object({
  nombre_completo: z
    .string()
    .min(2, "El nombre debe tener al menos 2 caracteres")
    .max(100, "El nombre no puede exceder 100 caracteres"),

  telefono: z
    .string()
    .max(20, "El teléfono no puede exceder 20 caracteres")
    .optional()
    .or(z.literal("")), // Permitir string vacío

  carrera: z
    .string()
    .max(100, "La carrera no puede exceder 100 caracteres")
    .optional()
    .or(z.literal("")),
})

// Tipo TypeScript generado automáticamente desde el esquema
export type PerfilFormData = z.infer<typeof perfilSchema>
```

Observa que primero creamos la carpeta `lib/validations/` para organizar todos los esquemas de validación del proyecto. A medida que avancemos, agregaremos esquemas para materias e inscripciones.

Nota: `.optional().or(z.literal(""))`

Los campos opcionales en HTML envían un string vacío ("") cuando el usuario no escribe nada. Sin `.or(z.literal(""))`, Zod rechazaría el string vacío porque espera `string | undefined`. Esta combinación acepta ambos.

Instalar el componente Form de shadcn/ui

shadcn/ui tiene un componente Form que se integra directamente con React Hook Form. Proporciona componentes como FormField, FormItem, FormLabel y FormMessage que conectan automáticamente los errores de validación con los inputs.

Terminal

```
npx shadcn@latest add form
```

Este comando instala el componente Form y sus dependencias. Genera el archivo components/ui/form.tsx que exporta:

Componente	Propósito
Form	Wrapper que conecta React Hook Form con el formulario HTML
FormField	Conecta un campo del formulario con React Hook Form (register)
FormItem	Contenedor para label + input + mensaje de error
FormLabel	Etiqueta del campo (se pone roja si hay error)
FormControl	Wrapper del input que conecta las props de accesibilidad
FormMessage	Muestra el mensaje de error de Zod automáticamente

Crear el componente de formulario de perfil

Este es el primer formulario completo con React Hook Form + Zod. Carga los datos actuales del usuario y permite editarlos.

Crea el archivo **components/dashboard/perfil-form.tsx**:

Archivo: components/dashboard/perfil-form.tsx

```
"use client"

import { useEffect, useState } from "react"
import { useForm } from "react-hook-form"
import { zodResolver } from "@hookform/resolvers/zod"
import { toast } from "sonner"

import { perfilSchema, type PerfilFormData } from "@/lib/validations/perfil"

import { Button } from "@/components/ui/button"
import { Input } from "@/components/ui/input"
import {
  Card,
  CardContent,
  CardDescription,
  CardHeader,
  CardTitle,
}
```

```

} from "@components/ui/card"
import {
  Form,
  FormControl,
  FormField,
  FormItem,
  FormLabel,
  FormMessage,
} from "@components/ui/form"

export function PerfilForm() {
  const [loading, setLoading] = useState(true)
  const [saving, setSaving] = useState(false)

  // 1. Configurar React Hook Form con Zod
  const form = useForm<PerfilFormData>({
    resolver: zodResolver(perfilSchema),
    defaultValues: {
      nombre_completo: "",
      telefono: "",
      carrera: "",
    },
  })

  // 2. Cargar datos actuales del perfil
  useEffect(() => {
    async function loadPerfil() {
      try {
        const response = await fetch("/api/auth/perfil")
        const data = await response.json()

        if (response.ok && data.perfil) {
          // Prellenar el formulario con datos existentes
          form.reset({
            nombre_completo: data.perfil.nombre_completo || "",
            telefono: data.perfil.telefono || "",
            carrera: data.perfil.carrera || "",
          })
        }
      } catch (error) {
        toast.error("Error al cargar el perfil")
      } finally {
        setLoading(false)
      }
    }

    loadPerfil()
  }, [form])

  // 3. Enviar datos actualizados
  async function onSubmit(values: PerfilFormData) {

```

```

    setSaving(true)

    try {
      const response = await fetch("/api/auth/perfil", {
        method: "PUT",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(values),
      })

      const data = await response.json()

      if (!response.ok) {
        toast.error(data.error || "Error al actualizar")
        return
      }

      toast.success("Perfil actualizado exitosamente")
    } catch {
      toast.error("Error de conexión")
    } finally {
      setSaving(false)
    }
  }

  if (loading) {
    return (
      <Card className="max-w-2xl">
        <CardContent className="p-6">
          <p className="text-muted-foreground">Cargando perfil...</p>
        </CardContent>
      </Card>
    )
  }

  return (
    <Card className="max-w-2xl">
      <CardHeader>
        <CardTitle>Mi Perfil</CardTitle>
        <CardDescription>
          Actualiza tu información personal
        </CardDescription>
      </CardHeader>
      <CardContent>
        <Form {...form}>
          <form onSubmit={form.handleSubmit(onSubmit)} className="space-y-6">
            /* — Nombre completo — */
            <FormField>
              control={form.control}
              name="nombre_completo"
              render={({ field }) => (
                <FormItem>

```

```

        <FormLabel>Nombre completo</FormLabel>
        <FormControl>
          <Input
            placeholder="Juan Pérez Mamani"
            {...field}
          />
        </FormControl>
        <FormMessage />
      </FormItem>
    )}
  />

  {/* — Teléfono — */}
  <FormField
    control={form.control}
    name="telefono"
    render={({ field }) => (
      <FormItem>
        <FormLabel>Teléfono (opcional)</FormLabel>
        <FormControl>
          <Input
            placeholder="+591 7xxxxxxx"
            {...field}
          />
        </FormControl>
        <FormMessage />
      </FormItem>
    )}
  />

  {/* — Carrera — */}
  <FormField
    control={form.control}
    name="carrera"
    render={({ field }) => (
      <FormItem>
        <FormLabel>Carrera (opcional)</FormLabel>
        <FormControl>
          <Input
            placeholder="Ingeniería de Sistemas"
            {...field}
          />
        </FormControl>
        <FormMessage />
      </FormItem>
    )}
  />

  {/* — Botón guardar — */}
  <Button type="submit" disabled={saving}>
    {saving ? "Guardando..." : "Guardar Cambios"}
  </Button>

```

```

        </form>
      </Form>
    </CardContent>
  </Card>
)
}

```

Desglose del formulario con RHF + Zod

useForm() + zodResolver: La línea `resolver: zodResolver(perfilSchema)` conecta Zod con RHF. Ahora, cuando el usuario hace submit, RHF automáticamente valida los datos contra el esquema de Zod antes de llamar a `onSubmit`.

form.reset(): En el `useEffect`, después de cargar los datos del servidor, usamos `form.reset()` para prellenar el formulario con los valores actuales. Esto es diferente a `setDefaultValue` — `reset` también limpia el estado de errores y `dirty fields`.

FormField + render: Cada campo usa el patrón `render prop`. El `{ field }` contiene `value`, `onChange`, `onBlur` y `ref` — todo lo que el `Input` necesita. El `{...field}` los pasa de golpe.

FormMessage: Este componente es mágico — automáticamente muestra el mensaje de error de Zod para ese campo específico. No necesitas escribir lógica condicional.

toast de Sonner: Importamos `toast` de `sonner` (no del componente). `toast.success()` y `toast.error()` muestran notificaciones con colores verde y rojo respectivamente.

Tip: ¿Por qué Form {...form}?

La línea `<Form {...form}>` pasa todo el contexto de `React Hook Form` al `provider` de `shadcn`. Esto permite que `FormField`, `FormLabel` y `FormMessage` accedan al estado del formulario sin que tú tengas que pasarlo manualmente.

Actualizar las páginas de perfil

Ahora reemplazamos los placeholders de perfil con el formulario real. Como el formulario es un `Client Component`, las páginas solo lo importan y renderizan.

Actualiza `app/(dashboard)/estudiante/perfil/page.tsx`:

```

Archivo: app/(dashboard)/estudiante/perfil/page.tsx
import { PerfilForm } from "@components/dashboard/perfil-form"

export default function PerfilEstudiante() {
  return (
    <div className="space-y-6">
      <h1 className="text-3xl font-bold">Mi Perfil</h1>
      <PerfilForm />
    </div>
  )
}

```

```
}
```

Actualiza `app/(dashboard)/docente/perfil/page.tsx` con el mismo contenido (cambiando el nombre de la función a `PerfilDocente`).

Nota: Componente compartido

El mismo `PerfilForm` funciona para estudiante y docente porque la API `/api/auth/perfil` identifica al usuario por su sesión (cookie), no por la URL. Un solo componente, múltiples páginas.

Crear el componente `StatsCard`

Vamos a crear un componente reutilizable para mostrar estadísticas en los dashboards. Lo usaremos extensamente en las entregas finales.

Crea el archivo `components/dashboard/stats-card.tsx`:

```
Archivo: components/dashboard/stats-card.tsx
import { type LucideIcon } from "lucide-react"
import {
  Card,
  CardContent,
  CardHeader,
  CardTitle,
} from "@components/ui/card"

interface StatsCardProps {
  title: string
  value: string | number
  description?: string
  icon: LucideIcon
}

export function StatsCard({
  title,
  value,
  description,
  icon: Icon,
}: StatsCardProps) {
  return (
    <Card>
      <CardHeader className="flex flex-row items-center justify-between space-y-0 pb-2">
        <CardTitle className="text-sm font-medium">
          {title}
        </CardTitle>
        <Icon className="h-4 w-4 text-muted-foreground" />
      </CardHeader>
      <CardContent>
        <div className="text-2xl font-bold">{value}</div>
        {description && (
```

```

        <p className="text-xs text-muted-foreground">
          {description}
        </p>
      )}
    </CardContent>
  </Card>
)
}

```

Ejemplo de uso del StatsCard

Ejemplo de uso (referencia)

```

// En cualquier página de dashboard:
import { BookOpen, Users, ClipboardList } from "lucide-react"
import { StatsCard } from "@/components/dashboard/stats-card"

// Dentro del JSX:
<div className="grid gap-4 md:grid-cols-3">
  <StatsCard
    title="Materias Inscritas"
    value={5}
    description="Semestre 2026-I"
    icon={BookOpen}
  />
  <StatsCard
    title="Créditos Totales"
    value={25}
    description="De 30 permitidos"
    icon={ClipboardList}
  />
  <StatsCard
    title="Promedio General"
    value="78.5"
    description="+2.3 vs semestre anterior"
    icon={Users}
  />
</div>

```

Patrón **icon**: **Icon**: Recibimos el icono como prop con nombre en minúscula (icon) y lo renombramos a mayúscula (Icon) en la desestructuración. React requiere que los componentes empiecen con mayúscula para distinguirlos de elementos HTML.

Crear la carpeta de validaciones

Si aún no la creaste, asegúrate de crear la carpeta para los esquemas de validación:

Terminal

```
mkdir -p lib/validations
```

Probar el formulario de perfil

1. Ejecuta `npm run dev`
2. Inicia sesión con cualquier usuario de prueba
3. Navega a **Mi Perfil** desde el sidebar
4. Verifica que los datos actuales se cargan en el formulario
5. **Prueba la validación:** borra el nombre y haz clic en "Guardar Cambios". Deberías ver el mensaje de error de Zod debajo del campo.
6. **Prueba la actualización:** cambia tu nombre y haz clic en guardar. Deberías ver un toast verde "Perfil actualizado exitosamente".
7. Recarga la página y verifica que los cambios persistieron
8. Verifica en Supabase Dashboard > Table Editor > usuarios_perfil que los datos cambiaron

Verificación Final

- ✓ `lib/validations/perfil.ts` exporta `perfilSchema` y `PerfilFormData`
- ✓ Componente Form de shadcn instalado (`components/ui/form.tsx` existe)
- ✓ El formulario de perfil carga los datos actuales al abrir la página
- ✓ La validación muestra errores debajo del campo correspondiente
- ✓ Guardar cambios muestra un toast de éxito y persiste en la BD
- ✓ El mismo PerfilForm funciona en `/estudiante/perfil` y `/docente/perfil`
- ✓ StatsCard se renderiza correctamente (probar con el ejemplo de uso)

Ejercicio Propuesto

9. **Validación de teléfono:** Modifica el esquema Zod para que el teléfono, si se proporciona, solo acepte números y el símbolo +. Pista: usa `.regex()` de Zod.
10. **Campo de solo lectura:** Agrega el email del usuario al formulario como un campo deshabilitado (solo lectura). Pista: usa el hook `useUsuario` para obtener el email y el prop `disabled` en el Input.
11. **Esquema compartido:** Importa `perfilSchema` en la API Route `app/api/auth/perfil/route.ts` y úsalo para validar el body del request en lugar de la validación manual. Pista: usa `perfilSchema.safeParse(body)`.

12. **Pregunta de reflexión:** ¿Cuál es la ventaja de definir el esquema de validación en un archivo separado (`lib/validations/perfil.ts`) en lugar de directamente en el componente? ¿En qué otros lugares podríamos reutilizarlo?
-

Próximo: Entrega 4B

En la siguiente entrega construiremos el panel de administración de usuarios: API Routes para listar y modificar usuarios, tabla con `shadcn/ui` con filtros por rol y estado, y acciones de activar/desactivar y cambiar rol con diálogos de confirmación.